SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

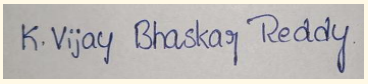
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	**ITA05**	**Course Title:**	**Computer Vision**
CO Assessed:	**CO5 – Naturalization (independent application using OpenCV)P5**	**Assessment:**	**Assessment Tool 2 – Pseudocode Writing Task**
Weightage:	**25% of CIA**	**Total Marks:**	**100**
CO4 Statement:	*Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL6)*		
Student Name:	K.Vijay Bhaskar Reddy	**Reg. No.:**	192425155
Section / Batch:	AI&ML/2024	**Date:**	25/08/2026

Code of Conduct

I K.Vijay Bhaskar Reddy(192425155) Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Criteria	Weigh tage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	

2. OCR Pipeline for Document Processing
A document processing system must:
A. Read input images containing printed text
B. Enhance image quality (noise removal, contrast improvement)
C. Segment text regions
D. Extract and display recognized text
Constraints:

● Input images may be noisy and low contrast
● Batch processing required

1. Problem Understanding

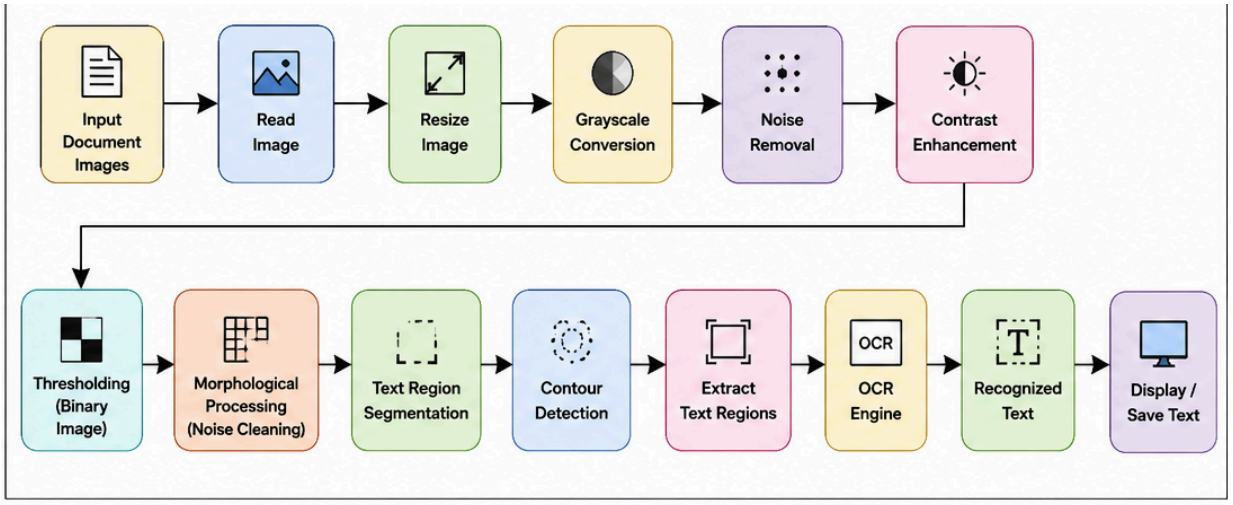
The objective is to design an **end-to-end OCR (Optical Character Recognition) pipeline** using OpenCV that can process document images containing printed text.

The system must:

- Read input document images.
- Improve image quality using preprocessing techniques.
- Remove noise and improve contrast.
- Segment text regions from the document.
- Extract recognized text using an OCR engine.
- Display the extracted text.
- Support **batch processing** of multiple images.

The pipeline should be robust against **noisy and low-contrast input images**.

2. Proposed OCR Pipeline



3. Pseudocode

```
BEGIN OCR_PIPELINE

   DEFINE input_folder
   DEFINE output_folder

   GET list_of_images from input_folder

   FOR each image_file in list_of_images DO

      PRINT "Processing:", image_file

      ------------------------------------------------
      STAGE 1: IMAGE INPUT
      ------------------------------------------------

      image ← cv2.imread(image_file)

      IF image is None THEN
         PRINT "Unable to read image"
         CONTINUE
      END IF


      ------------------------------------------------
      STAGE 2: IMAGE PREPROCESSING
      ------------------------------------------------

      resized_image ← cv2.resize(
         image,
         required_size
      )

      gray_image ← cv2.cvtColor(
         resized_image,
         cv2.COLOR_BGR2GRAY
      )

      denoised_image ← cv2.GaussianBlur(
         gray_image,
         (5,5),
         0
      )


      ------------------------------------------------
      STAGE 3: CONTRAST ENHANCEMENT
      ------------------------------------------------
```

```
    contrast_image ← CLAHE(
        denoised_image
    )

    enhanced_image ← contrast_image.apply(
        denoised_image
    )


    ------------------------------------------------
    STAGE 4: TEXT SEGMENTATION
    ------------------------------------------------

    binary_image ← cv2.threshold(
        enhanced_image,
        0,
        255,
        cv2.THRESH_BINARY + cv2.THRESH_OTSU
    )

    kernel ← cv2.getStructuringElement(
        cv2.MORPH_RECT,
        (3,3)
    )

    cleaned_image ← cv2.morphologyEx(
        binary_image,
        cv2.MORPH_OPEN,
        kernel
    )

    cleaned_image ← cv2.morphologyEx(
        cleaned_image,
        cv2.MORPH_CLOSE,
        kernel
    )


    ------------------------------------------------
    STAGE 5: TEXT REGION DETECTION
    ------------------------------------------------

    contours ← cv2.findContours(
        cleaned_image,
        cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE
    )

    text_regions ← EMPTY LIST
```

```
    FOR each contour in contours DO

        area ← cv2.contourArea(contour)

        IF area > minimum_area THEN

            x,y,w,h ← cv2.boundingRect(
               contour
            )

            IF width and height satisfy
              text_region_conditions THEN

                ADD (x,y,w,h)
                TO text_regions

            END IF

        END IF

    END FOR


    ------------------------------------------------
    STAGE 6: SORT TEXT REGIONS
    ------------------------------------------------

    SORT text_regions
    from top-to-bottom
    and left-to-right


    ------------------------------------------------
    STAGE 7: OCR TEXT EXTRACTION
    ------------------------------------------------

    recognized_text ← EMPTY STRING

    FOR each region in text_regions DO

        x,y,w,h ← region

        text_image ← crop(
            enhanced_image,
            x,y,w,h
        )

        text ← OCR(
            text_image
        )
```

```
        recognized_text ←
            recognized_text + text

    END FOR

    ---------------------------------------------
    STAGE 8: DISPLAY RECOGNIZED TEXT
    ---------------------------------------------

    PRINT "Recognized Text:"
    PRINT recognized_text


    ---------------------------------------------
    STAGE 9: SAVE RESULT
    ---------------------------------------------

    output_file ← create_output_filename(
        image_file
    )

    WRITE recognized_text
    INTO output_file

    DISPLAY cleaned_image

  END FOR

  PRINT "Batch OCR processing completed."

END OCR_PIPELINE
```

4. OpenCV Functions Used

Stage	OpenCV Function	Purpose
Image input	cv2.imread()	Reads document image
Resizing	cv2.resize()	Standardizes image size
Grayscale	cv2.cvtColor()	Converts image to grayscale
Noise removal	cv2.GaussianBlur()	Removes image noise
Contrast	cv2.createCLAHE()	Improves local contrast
Thresholding	cv2.threshold()	Converts image to binary

Morphology	cv2.morphologyEx()	Removes noise and connects text regions
Contours	cv2.findContours()	Detects possible text regions
Bounding boxes	cv2.boundingRect()	Identifies text-region coordinates

5. Control Structures

The pseudocode uses several control structures required for batch processing.

FOR loop

The FOR loop processes every image in the input directory:

FOR each image_file in list_of_images DO

This enables **batch OCR processing**.

IF condition

The IF statement checks whether an image was successfully loaded:

```
IF image is None THEN
   PRINT "Unable to read image"
   CONTINUE
END IF
```

It prevents the pipeline from stopping because of one invalid image.

Nested FOR loop

Another FOR loop processes each detected text region:

FOR each region in text_regions DO

This allows individual regions to be passed to the OCR engine.

6. Handling Noisy and Low-Contrast Images

Noise Removal

Gaussian filtering is applied before segmentation:

```
denoised_image ← cv2.GaussianBlur(
   gray_image,
   (5,5),
   0
)
```

This reduces random noise and small artifacts that could otherwise be incorrectly detected as text.

Contrast Enhancement

CLAHE improves local contrast:

```
contrast_image ← CLAHE(
   denoised_image
)
```

This is useful when text is faint or the document contains uneven illumination.

Thresholding

Otsu thresholding automatically determines a suitable threshold:

```
binary_image ← cv2.threshold(
   enhanced_image,
   0,
   255,
   cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
```

This converts the document into a form where text can be separated from the background.

Morphological Processing

Opening removes small noise, while closing connects broken portions of characters and text regions.

7. Text Segmentation

After preprocessing, contours are detected:

```
contours ← cv2.findContours(
   cleaned_image,
   cv2.RETR_EXTERNAL,
   cv2.CHAIN_APPROX_SIMPLE
)
```

For every contour, the area and bounding rectangle are calculated.

Small contours are discarded:

IF area > minimum_area THEN

This reduces false text detections caused by noise.

The remaining bounding boxes represent potential **text regions**.

8. Text Extraction

Each detected text region is cropped and passed to an OCR engine:

```
text_image ← crop(
   enhanced_image,
   x,y,w,h
)

text ← OCR(
   text_image
)
```

The recognized text is then combined:

```
recognized_text ←
   recognized_text + text
```

Finally, the extracted text can be displayed on the screen and saved into a text file.

9. Batch Processing

The pipeline supports batch processing by obtaining all images from the input folder:

GET list_of_images from input_folder

FOR each image_file in list_of_images DO

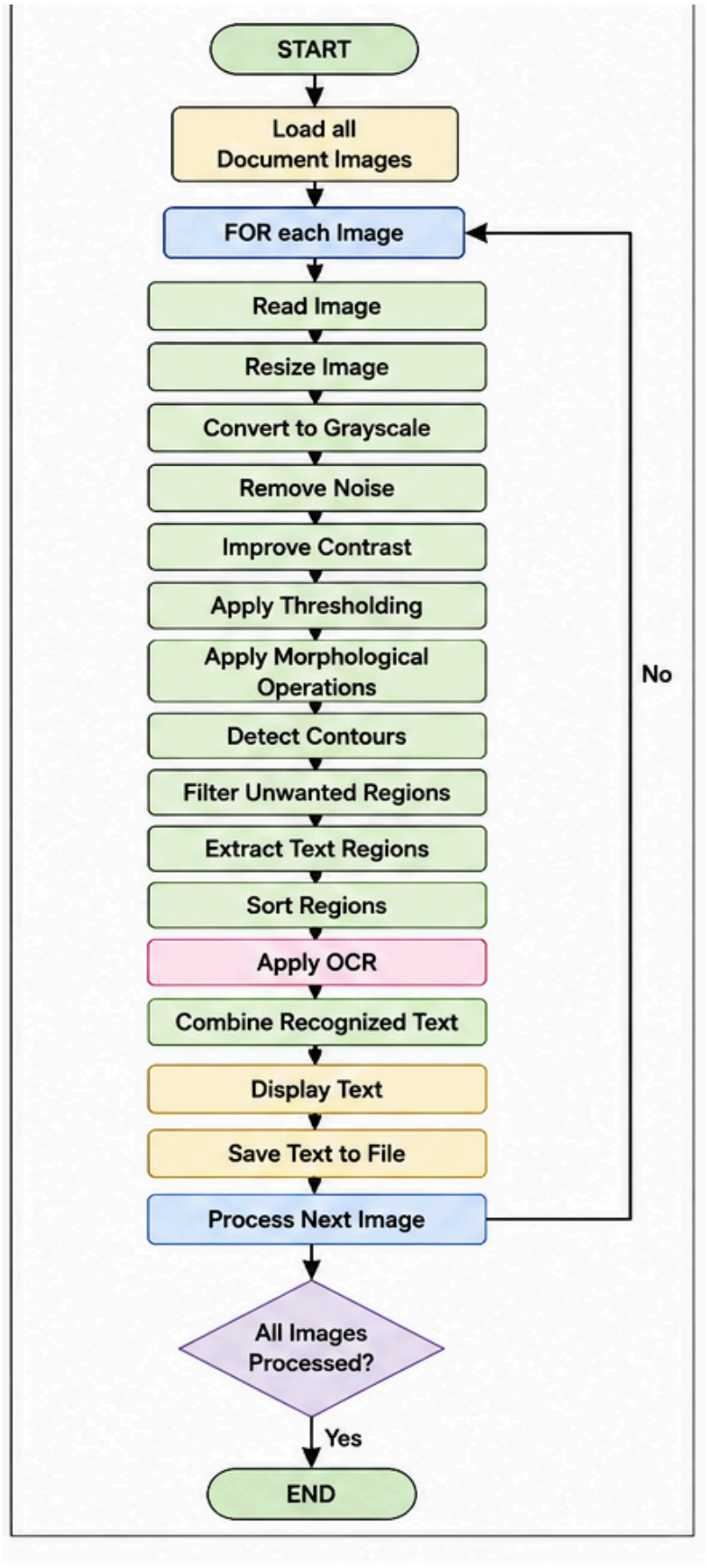
For example:

```
Input/
├── document1.jpg
├── document2.jpg
├── document3.jpg
├── document4.png
└── document5.jpg
```

The system processes each image automatically and generates corresponding output files:

```
Output/
├── document1.txt
├── document2.txt
├── document3.txt
├── document4.txt
└── document5.txt
```

10. Overall Algorithm



11. Conclusion

The proposed OCR pipeline provides a complete solution for **batch document processing** using OpenCV. Image preprocessing improves the quality of noisy and low-contrast documents, while thresholding and morphological operations make text segmentation more reliable. Contour detection identifies potential text regions, and an OCR engine extracts the recognized characters. The use of **loops, conditional statements, image-processing functions, segmentation, and OCR** makes the pipeline suitable for automated processing of multiple document images.